FlowRefine

Can visual feedback improve generated flowcharts on a CPU?

Generative AI final project | Reproducible pilot study | 24 October 2026

Research question

Does one round of feedback-driven editing improve the semantic correctness and layout of editable flowcharts? We compare programmatic, visual, and combined feedback under the same initial diagram and the same text editor. The study measures both correction and regression; it does not assume that adding a visual critic helps.

Observed outcome: the initial model produces exactly correct graphs on 11/15 tasks. Exact correctness after programmatic, visual and combined feedback is 11/15, 9/15 and 7/15. No feedback arm improves mean edge F1 in this run. The result argues for checking regressions before trusting a small visual critic, within this limited model and task setting.

System and contributions

A text model converts an explicit process description into a compact JSON graph. Each node contains its exact stage label, type and grid location; each edge contains source label, target label and branch label. A deterministic renderer expands the grid into fixed-size blocks and routes arrows, producing editable SVG and a PNG for visual inspection. Grid coordinates simplify output syntax but restrict possible layouts.

A reference-free checker identifies geometry and rendering risks. A separate small vision-language model receives only the rendered image and the original request. The text editor then regenerates the complete graph using the assigned feedback. No critic, generator or editor receives a reference graph, semantic score or test-ground-truth correction.

Project contributions are an implemented scene representation and renderer; geometric checks and structured programmatic feedback; an image-conditioned criticism and editing pipeline; a small controlled dataset; and auditable semantic, geometry and inference-cost analysis. Pretrained models are reused without fine-tuning. The contribution is a course-level system and evaluation, not a claim of a novel model architecture.

Dataset and scope

The dataset contains 5 development tasks and 15 held-out test tasks: five chains, five two-way branches and five loops. Each process contains 6 or 7 explicitly named nodes. A fixed template verbalizes assistant-authored process specifications. Development and test topics differ; graph families remain shared. This evaluates unseen process content within familiar structures, not transfer to unseen graph topologies or unrestricted natural-language workflows.

All prompts request exact labels and explicit Yes/No decision branches. References contain nodes, types and directed labeled edges, but no target coordinates. Generation reads public task files; references are read only by the analysis stage. No independent human-authored or blinded evaluation dataset is claimed.

Frozen experimental protocol

Arm	Available feedback for one editing round
B0	Initial generated graph; no editing.
B1	Reference-free programmatic geometry checks.
B2	Visual critic output from the initial PNG.
B3	The same programmatic checks and visual critic output.

Every task has one shared initial graph. B1-B3 each use one semantic editing round, with a common text model and prompt format. The VLM feedback is computed once per initial image and reused identically by B2 and B3. This is shared feedback reuse, not a second independent visual sample. A bounded format-recovery retry may occur after invalid output; these calls are logged and included in costs. No best-of-test-score selection is used.

CPU implementation and model settings

Setting	Recorded value
Text generator/editor	qwen2.5-3b-instruct-q4_k_m.gguf
Visual critic	SmolVLM-256M-Instruct-Q8_0.gguf
Vision projector	mmproj-SmolVLM-256M-Instruct-f16.gguf
Backend	llama-cpp-python 0.3.36; MTMD vision; CPU only
Decoding	temperature=0.0; seed=241024; threads=4
Output budgets	text=900; vision=180; format retries=1
Prompt version	compact-grid-v3

Models are loaded sequentially without GPU offloading. Exact repositories, revisions, byte sizes and SHA-256 hashes are recorded in configs/model_lock.json; runtime_setup/HANDOFF.md documents CPU smoke tests. Qwen2.5-3B uses the qwen-research license; SmolVLM distributions use Apache-2.0. Initial 1.5B probes are exploratory and retained under development runs, not pooled into test metrics. The larger text model was selected on development tasks. Earlier smoke scripts used an older output format. Prompt or model changes were development decisions, not per-test-task selection.

Metrics and failure handling

Nodes match one-to-one by normalized exact label and type. Normalization applies Unicode NFKC, case folding and punctuation-to-space replacement. Edges match only when source, target, direction and normalized branch label agree. Node and edge precision, recall and F1 penalize omissions and additions. Exact semantics requires a complete match. No fuzzy matcher or model judge is used.

Executed invalid scenes receive zero semantic scores. Unexecuted/unavailable arms remain missing and are excluded from quality means. Geometry counts are defined only on valid scenes, with coverage reported alongside them; invalid scenes are never interpreted as zero geometry errors. Render availability is tracked separately. Paired differences use comparable tasks and a fixed-seed 1,000-resample task bootstrap.

Results from saved inference

Run: test_v1. Public tasks: 15. Provenance: real_model_inference. All values below are read from saved analysis artifacts; no expected or illustrative accuracy is inserted.

Arm	Run / valid	Node F1	Edge F1	Exact	Seconds/task
B0	15 / 15	1.000	0.957	0.733	16.5
B1	15 / 15	1.000	0.941	0.733	37.1
B2	15 / 15	1.000	0.912	0.600	48.6
B3	15 / 15	1.000	0.899	0.467	50.2

Run / valid gives executed and structurally valid counts. Scores average all executed tasks, including invalid outputs as zero. Exact is the proportion of exactly correct graphs. Seconds/task is standalone model-call time including shared generation and the relevant visual critique.

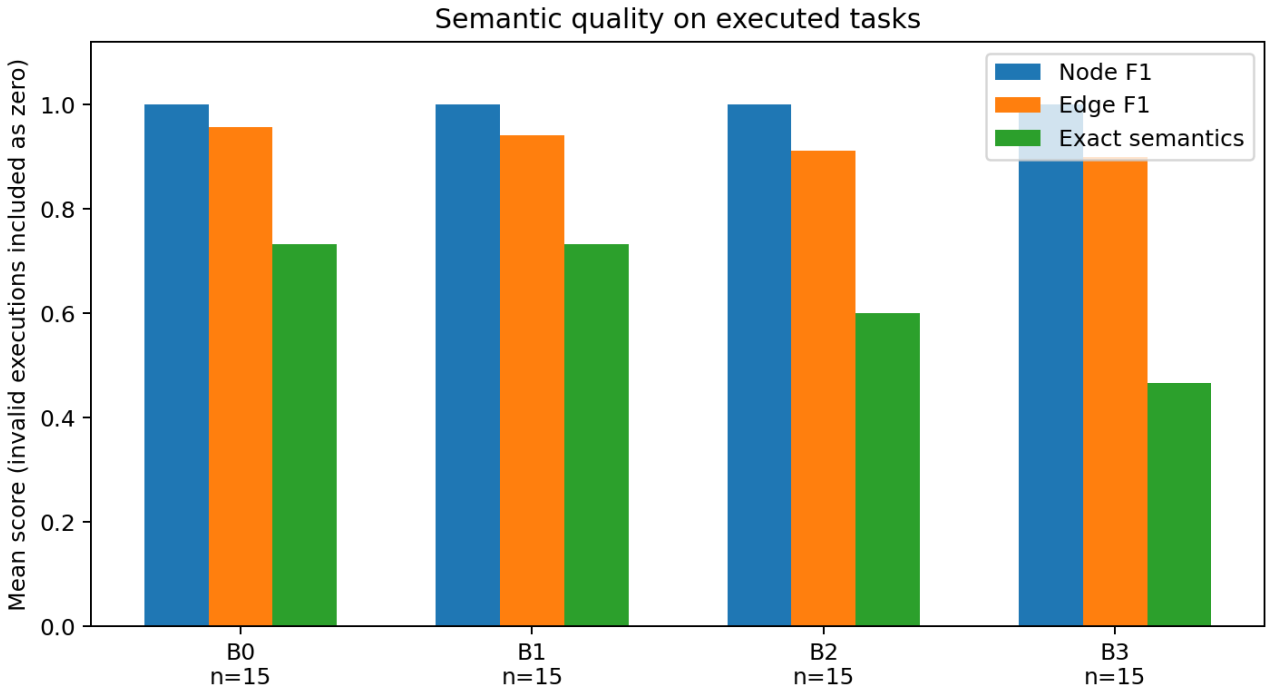


Figure 1. Semantic quality by feedback arm. Sample counts are shown; higher is better.

Paired semantic effects

B1 minus B0: mean edge-F1 change -0.015; bootstrap 95% interval [-0.062, 0.015]; paired tasks n=15.

B2 minus B0: mean edge-F1 change -0.045; bootstrap 95% interval [-0.101, -0.006]; paired tasks n=15.

B3 minus B0: mean edge-F1 change -0.058; bootstrap 95% interval [-0.104, -0.018]; paired tasks n=15.

Intervals are descriptive for this small pilot and are not multiplicity-corrected. Arm means may use different task sets when an arm is unavailable; paired effects are the appropriate within-task comparison. Exact scores and edge F1 should be considered together with validity and geometry.

For this run, no feedback arm improves mean edge F1 over the shared initial baseline. This is a limited empirical outcome, not evidence that stronger visual critics cannot help.

Layout, cost and interpretation

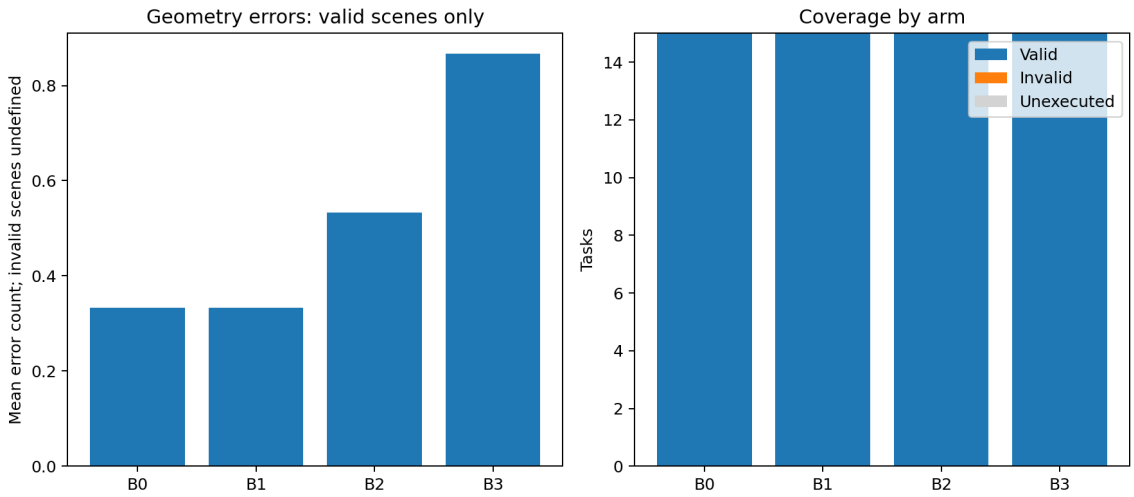


Figure 2. Geometry error counts are conditional on validity; coverage includes invalid and unavailable cases.

Actual persisted calls: 75; calls explicitly marked cached: 0. Total logged model time: 1312.9 seconds. Logged prompt/completion tokens: 51263 / 10386. Each saved call is counted once in the actual total. Standalone arm costs intentionally reinclude shared dependencies and must not be summed as the physical experiment cost.

Feedback reuse across B2/B3 is distinct from a backend cache hit. Timings are serial single-pass observations: the first call may include model loading, and prompt-prefix caches may be warm. They are not a standardized serving benchmark. Local CPU checking/rendering is excluded. Token counts are recorded from model responses. See raw_calls.csv for per-call evidence and quality_cost.png for the quality/time plot.

Limitations and next steps

Fifteen synthetic test tasks, one decoding seed and shared graph templates do not support broad generalization. Strict labels can penalize meaningful paraphrases. Fixed grid geometry and automatic arrow routing constrain what the editor can change. The tiny visual model may describe a diagram instead of diagnosing errors, hallucinate links or produce unhelpful criticism. A null or negative repair effect is a result about this implementation and model budget, not proof that visual feedback cannot work.

There is no independent human or blinded readability study: any qualitative inspection is assistant spot-checking only. We do not estimate human preference. Stronger follow-up work would add an equal-budget resampling control, a standard graph-layout baseline, multiple seeds, a stronger VLM, free-form descriptions and independent human evaluation. These are not represented as completed experiments.

Related work, sources and AI assistance

IntroSVG [1] studies generator-critic feedback for text-to-SVG and includes learned alignment. This project uses pretrained models and checks directed process semantics; it does not reproduce IntroSVG training or claim novelty for the generator-critic idea. The CPU models/runtime are documented in [2-5]. Codex assistance was used for implementation, synthetic-task authoring, execution, evaluation and report drafting. All reported measurements derive from persisted inference logs.

[1] Wang et al. IntroSVG: Learning from Rendering Feedback for Text-to-SVG Generation via an Introspective Generator-Critic Framework (2026). https://arxiv.org/abs/2603.09312

[2] Qwen model source and GGUF distributions. https://huggingface.co/Qwen/Qwen2.5-3B-Instruct-GGUF (exact checkpoint, license and revision in configs/model_lock.json).

[3] SmolVLM-256M-Instruct model card. https://huggingface.co/HuggingFaceTB/SmolVLM-256M-Instruct

[4] CPU GGUF conversion and projector. https://huggingface.co/ggml-org/SmolVLM-256M-Instruct-GGUF

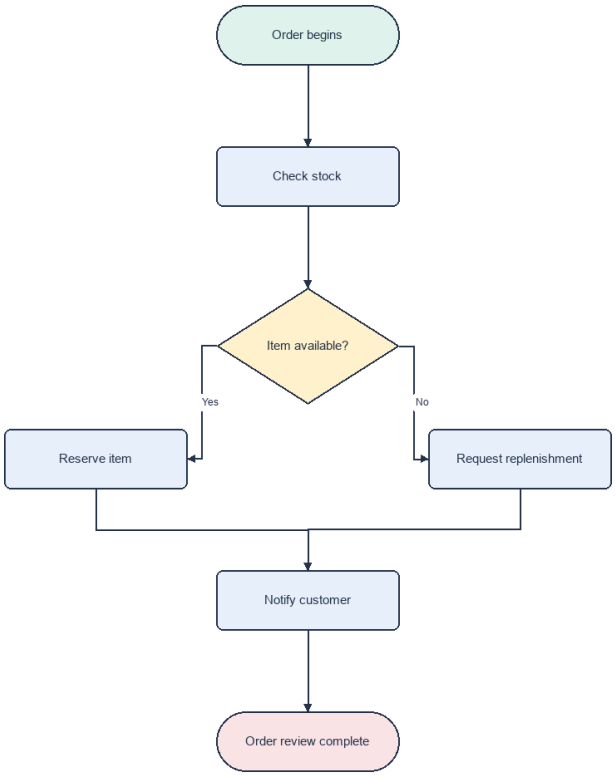
[5] llama.cpp runtime. https://github.com/ggml-org/llama.cpp ; Python bindings: https://github.com/abetlen/llama-cpp-python

Appendix: one saved before/after case

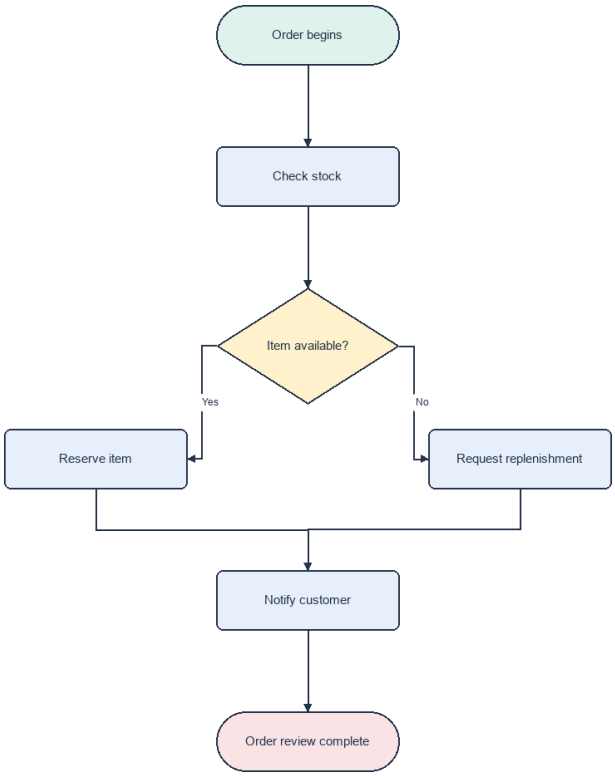
Task test_branch_01: selected as the first task ID with valid rendered B0 and B3 outputs. It is not selected for the largest improvement. This is an illustrative case, not an independent human quality judgment. Empty margins are cropped; clipping already present in an original rendered canvas is preserved.

Create a readable directed flowchart for the following process. Use exactly the quoted stage labels. The first stage is a start node, the last is an end node, any explicitly named decision is a decision node, and all other stages are process nodes. Label decision branches exactly Yes and No. Do not add stages or connections. Begin with "Order begins", then "Check stock", then the decision "Item available?". Its Yes branch goes to "Reserve item"; its No branch goes to "Request replenishment". Both alternative stages lead directly to "Notify customer", which leads to "Order review complete".

B0 initial



B3 combined feedback



Saved visual feedback (verbatim; may be incorrect):

Order Begins is connected with Check stock which is then connected with Item available? If Item available? is No then Reserve item and if Item available? is Yes then Request replenishment and if Item available? is Yes then Item available? Also, Reserve item is connected with both Order review complete and Order review complete which is then connected with both Order review complete and Order review complete. If Reservation dey is No then Reserve item and if Reservation dey is Yes then Request replenishment and if Reservation dey is No then Order review complete.